

Why Servey (WebRTC) Needs STUN/TURN

How peer-to-peer screen streaming connects across networks — and how Macky and similar apps really work. Prepared for the Servey project.

1. What Macky actually does

Macky uses the same design as Servey: a **WebRTC peer-to-peer (P2P) connection** with a **signaling server** that introduces the two devices but does not carry the media stream (this is exactly the role Firestore plays in Servey). In the author's own words:

"My server introduces the devices but never sees the data stream... it establishes a direct peer-to-peer connection using WebRTC... Your terminal data never touches our servers."

The claim is about the **signaling server** — it never sees the video/terminal bytes. That is already true for Servey too. The important nuance the marketing omits: **"never touches our servers" really means "we cannot read your data"** (it is end-to-end encrypted), not that the bytes never pass through any relay. See section 4.

2. The basics: why P2P is hard (NAT)

Your Mac and iPad have no public internet address. They sit behind a router doing **NAT** (Network Address Translation): the Mac is a private `192.168.x.x`, and the outside world only sees the router's public IP. So when the iPad tries to send video "to the Mac," there is no public address to aim at, and the router drops unsolicited incoming packets. Both devices have this problem at the same time.

WebRTC solves this with **ICE**, which gathers "candidates" (possible paths):

Candidate	How it is found	Works when...
host	The device's own LAN IP	Both devices on the same network (the cable test) ✓
srflx (server-reflexive)	A STUN server reports your public IP:port	Routers use "friendly" NAT (~75–80% of networks)
relay	A TURN server relays the traffic	Always — the fallback for the remaining ~20–25%

3. STUN vs TURN

STUN (what Servey has now — `stun.l.google.com`) is a cheap, free "mirror." The Mac asks "what is my public address?" and uses it to attempt a direct hole-punch to the iPad. **No data flows through STUN** — it is only a lookup. This is enough for most home WiFi.

TURN is an actual **relay**. When a direct hole-punch is impossible, both devices send their media to the TURN server, which forwards it to the other side. Data **does** flow through TURN — but it stays **end-to-end encrypted** (WebRTC DTLS-SRTP). The TURN server relays **encrypted blobs it cannot decrypt**.

4. The crux of the question

Even when a product uses TURN, it can **honestly** say "we cannot see your data" / "end-to-end encrypted," because the relay only ever handles ciphertext. That is almost certainly how Macky operates: **P2P when possible, encrypted TURN relay when not**, while truthfully claiming it cannot read your stream.

5. Do you actually need TURN? (real-world data)

From Chrome's real-world telemetry (UMA) and multiple industry sources:

- ~**75–80%** of connections succeed with **STUN only** (direct P2P).
- ~**20–25%** *require* a **TURN relay**.
- On **cellular / CGNAT / corporate** networks (symmetric NAT) this rises to **25–50%**.
- Two **symmetric NATs** can **never** be hole-punched — TURN is the only path.

Scenario	What happens
Same WiFi (no cable)	Usually works with STUN, sometimes host candidates — no TURN needed
Different networks / iPad on cellular / strict router	STUN fails — TURN is mandatory

Survey's failed cross-network test landed in the ~20–25% that needs a relay. **Macky works "anywhere" precisely because it runs a TURN server** for that fraction. A pure no-TURN app silently fails for those same users.

6. Am I 100% sure Macky uses TURN?

I cannot audit Macky's private servers, so not literal 100% on *their* infrastructure — but on the **technical necessity I am ~99% certain**:

- Macky advertises working on **cellular** ("from the toilet / bar / bed").
- Cellular uses symmetric NAT/CGNAT; hole-punching through two symmetric NATs is a **mathematical impossibility**.
- So a cellular-capable app **must** either (a) run TURN, or (b) silently fail for 20–50% of users.
- Macky works on cellular without mass "won't connect" complaints ⇒ **(a) is effectively guaranteed**.

7. Bottom line

- TURN does **not** weaken the "P2P, end-to-end encrypted" story. When P2P works, TURN is not used at all.
- When TURN is needed, it only relays encrypted bytes it cannot read — exactly like Macky.
- For Survey to work **reliably from anywhere**, a TURN server is required. For LAN-only use, it is not.

Sources: Macky author (Hacker News / App Store); Chrome UMA NAT-traversal telemetry; RFC 8835 (Transports for WebRTC); getstream.io, liveapi.com, celloip.com TURN analyses. Generated for the Survey project.